Alma Mater Studiorum-Università di Bologna
Scuola di Ingegneria

# Il tema della genericità: tipi generici e polimorfismo orizzontale

*Corso di Laurea in Ingegneria Informatica*

Anno accademico 2025/2026

**Prof. ENRICO DENTI**

*Dipartimento di Informatica – Scienza e Ingegneria (DISI)*

ALMA MATER STUDIORUM - UNIVERSITÀ DI BOLOGNA
IL PRESENTE MATERIALE È RISERVATO AL PERSONALE DELL'UNIVERSITÀ DI BOLOGNA E NON PUÒ ESSERE UTILIZZATO AI TERMINI DI LEGGE DA ALTRE PERSONE O PER FINI NON ISTITUZIONALI

# IL TEMA DELLA GENERICITÀ

- Per genericità si intende la possibilità di definire componenti software *generici rispetto al tipo*
  - strutture dati (array, liste, alberi, tabelle..) usabili su qualunque tipo di oggetto
  - funzioni / metodi che accettano qualunque tipo di oggetto
- *Un modo per farlo è sfruttare il tipo* **`Object`**, che è il «tipo più generale possibile», compatibile con *ogni altro*
  - strutture dati in cui il tipo dell'elemento sia **`Object`**
  - funzioni che accettano un argomento di tipo **`Object`** e/o restituiscono un risultano di tipo **`Object`**
- NB: a parte i tipi primitivi, da incapsulare nei rispettivi wrapper ☹

# STRUTTURE DATI GENERICHE con `Object`

- ESEMPIO: la lista generica «a puntatori», **`edutils.List`**

Java

```
package edutils;

public class List {
 private Object elem;
 private List next;

 public List(){ elem=null; next=null;}
 public List(Object e){ elem=e; next=null; }
 public List(Object e, List l){ elem=e; next=l; }

 public Object head(){ return elem; }
 public List tail(){ return next; }

 public boolean isEmpty(){ return elem==null; }

 public String toString(){ return isEmpty() ? "" :
   elem.toString() + ((next==null || (next!=null &&
       next.isEmpty())) ? "" : ", " + next.toString()); }

}
```

Il tipo effettivo dell'elemento può essere *un qualsiasi Object* (ma non un tipo primitivo, perché essi non sono oggetti)

# STRUTTURE DATI GENERICHE con `Object`

- Uso inteso: poter creare specifiche liste di ciò che ci serve

Java

```
List l1 = new List();
List l2 = new List("Pippo",
              new List("Alberto",
                 new List("Enrico")) );
List l3 = new List( new Counter(2),
              new List(new Counter2(18)) );
List l4 = new List( new Frazione(2,3),
              new List(new Frazione(5,7)) );
System.out.println(l1);
System.out.println(l2);
System.out.println(l3);
System.out.println(l4);
```

Lista vuota

Lista di stringhe

Lista di Counter

Lista di frazioni

```
Pippo, Alberto, Enrico
Counter di valore 2, Counter2 di valore 18
2/3, 5/7
```

- NB: nelle intenzioni, ogni lista è *di un ben preciso tipo*

# FUNZIONI (LIBRERIE) GENERICHE con `Object`

- ESEMPIO: una funzione polimorfa che *cerchi se un array contenga un elemento «uguale» a un certo elemento*
- Perché operi su qualunque array, si può pensare di definirla in modo che accetti un *array di* **`Object`** *e un dato* **`Object`**

```
class Main {
 public static boolean find(Object[] array, Object elem){
  for (Object obj : array) if (obj.equals(elem)) return true;
  return false;
 }
}
```

Java

- Uso inteso:

```
String[]   arr1 = { "Pippo", "Alberto", "Enrico" };
Frazione[] arr2 = { new Frazione(2,3),new Frazione(5,7)};

var res1 = find(arr1, "Giusy");
var res2 = find(arr2, new Frazione(4,6));
```

# STRUTTURE DATI GENERICHE con `Object`

- Ma purtroppo è legale anche un mix come questo:

Java

```
List l5 = new List( new Frazione(2,3),
                    new List("Pippo",
                        new List(new Counter2(18))) );
System.out.println(l5);
```

```
Pippo, Alberto, Enrico
Counter di valore 2, Counter2 di valore 18
2/3, 5/7
2/3, Pippo, Counter2 di valore 18
```

- MOTIVO: definire strutture come «contenitori di `Object`» permette sì di ottenere contenitori «generici» (bene!)…
- …ma anche di metteci *dentro qualunque cosa (male!)*
- MORALE: usare `Object` per ottenere genericità implica <u>pagare un prezzo altissimo</u>: *abolire il controllo di tipo!*

# FUNZIONI (LIBRERIE) GENERICHE con `Object`

- Un problema analogo si presenta con le funzioni:
  - l'uso inteso era che **`elem`** fosse dello stesso tipo effettivo di **`array`**
  - MA tale vincolo è rimasto solo «nella testa del progettista»: *non si è tradotto in codice!*
  - quindi sono legali anche assurdità come queste:
- Un caso di mis-use:

```
String[]   arr1 = { "Pippo", "Alberto", "Enrico" };
Frazione[] arr2 = { new Frazione(2,3),new Frazione(5,7)};

var res1 = Main.find(arr1, new Counter(18));
var res2 = Main.find(arr2, "Alberto");
```

- Di nuovo: usare **`Object`** come «jolly» permette sì di scrivere funzioni generiche rispetto al tipo…
- …ma anche di *passare a run time qualunque cosa!* ☹

# GENERICITÀ: UN PRIMO BILANCIO

- Per questo motivo, l'uso di `Object` per scrivere componenti software *generici rispetto al tipo* è un approccio *obsoleto, insoddisfacente, error-prone*
  - si bypassa di fatto il controllo di tipo
  - si pongono le premesse per «esplosioni» a run-time dovute a errori non intercettati in fase di compilazione
- Il modo moderno per soddisfare la necessità di *genericità* è costituita dal concetto di *tipo parametrico (o generico)*
  - il tipo c'è, ma diventa un *parametro (eventualmente inferito)*
  - in tal modo diviene possibile *specificarlo al momento dell'uso* della funzione o del componente
  - RISULTATO: genericità senza compromessi ☺

# IL TIPO PARAMETRICO (GENERICO)

- La nozione di *tipo generico* consente di *usare un simbolo per il tipo,* che diviene un *parametro* da specificare poi, al *momento dell'uso*
  - in Java, C#, Kotlin: notazione **`<T>`**
  - in Scala: notazione **`[T]`**

L'identificatore può essere qualsiasi: i più usati sono **`T`** («Type») ed **`E`** («Element»)

- REFACTORING: riprogettiamo sia la lista, sia la funzione **`find`** utilizzando la moderna nozione di *tipo generico*
  - non più argomenti di tipo **`Object`**, ma di tipo generico **`T`**
  - volta per volta, *al momento dell'uso,* specificheremo *quale tipo* ci interessi realmente.

# STRUTTURE DATI GENERICHE CON TIPO PARAMETRICO

- Lista generica **`edutils2.List`** – nuova versione

Java

```
package edutils2;

public class List<T> {
 private T elem;
 private List<T> next;

 public List(){ elem=null; next=null;}
 public List(T e){ elem=e; next=null; }
 public List(T e, List<T> l){ elem=e; next=l; }

 public T head(){ return elem; }
 public List<T> tail(){ return next; }

 public boolean isEmpty(){ return elem==null; }

 public String toString(){ return isEmpty() ? "" :
   elem.toString() + ((next==null || (next!=null &&
       next.isEmpty())) ? "" : ", " + next.toString()); }

}
```

La lista è ora definita esplicitamente come lista «di **T**»

Tipo parametrico **T** al posto di **Object**

NB:nei costruttori non è previsto il tag **<T>**

# STRUTTURE DATI GENERICHE CON TIPO PARAMETRICO

- L'uso inteso rimane possibile, ma ora si deve specificare esattamente *il tipo di liste desiderato, caso per caso:*

```
List<Object> l1 = new List<Object>();
List<String> l2 = new List<String>("Pippo",
                 new List<String>("Alberto",
                     new List<String>("Enrico")) );
List<Counter> l3 = new List<Counter>( new Counter(2),
                 new List<Counter>(new Counter2(18)));
List<Frazione> l4 = new List<Frazione>(
                 new Frazione(2,3),
                 new List<Frazione>(new Frazione(5,7)) );
System.out.println(l1);
System.out.println(l2);
System.out.println(l3);
System.out.println(l4);
```

Lista di qualsiasi cosa

Java

Lista di stringhe

Lista di Counter

Lista di frazioni

```
Pippo, Alberto, Enrico
Counter di valore 2, Counter2 di valore 18
2/3, 5/7
```

# STRUTTURE DATI GENERICHE CON TIPO PARAMETRICO

- PUNTO CHIAVE: il mix incontrollato non è ora più possibile, perché le liste così ottenute sono *incompatibili* fra loro

```
l2 = l3; // NO!
l2 = l4; // NO!
l4 = l2; // NO!
```

- Per poter mettere oggetti «qualsiasi» (disomogenei) in una lista ora bisogna etichettarla *esplicitamente* come tale:

Java

```
List<Object> l5 = new List<Object>(
                      new Frazione(2,3),
                      new List<Object>("Pippo",
                        new List<Object>(new Counter2(18))
                      );

System.out.println(l5);
```

```
Pippo, Alberto, Enrico
Counter di valore 2, Counter2 di valore 18
2/3, 5/7
2/3, Pippo, Counter2 di valore 18
```

# FUNZIONI (LIBRERIE) GENERICHE CON TIPO PARAMETRICO

- La stessa cosa si può fare per le funzioni: anch'esse possono essere definite usando il segnaposto generico

DEFINIZIONE di funzione generica in tipo

Java

```
public static <T> boolean find(T[] array, T elem)
```

CHIAMATA di una funzione generica (es. con T=Counter)

```
nomeclasse.<Counter>idem(arr, e)
```

Java

- NB: in Java, la posizione del segnaposto in fase di chiamata è purtroppo un po' innaturale; altri linguaggi fanno scelte più felici

# FUNZIONI (LIBRERIE) GENERICHE CON TIPO PARAMETRICO

- Funzione generica find – nuova versione

```
public static <T> boolean find(T[] array, T elem){
   for (T obj : array)
    if (obj.equals(elem)) return true;
   return false;
}
```

Java

  - anche qui, non più argomenti di tipo **`Object`**, ma di tipo generico **`T`**
  - volta per volta, *al momento dell'uso,* specificheremo *quale tipo*

- Uso:

```
String[]   arr1 = { "Pippo", "Alberto", "Enrico" };
Frazione[] arr2 = { new Frazione(2,3),new Frazione(5,7)};

var res1 = Main.<String>find(arr1, "Giusy");
var res2 = Main.<Frazione>find(arr2, new Frazione(4,6));
```

qui, **`T = String`**

qui, **`T = Frazione`**

Posizione del tag un po' innaturale… ☹

# FUNZIONI (LIBRERIE) GENERICHE CON TIPO PARAMETRICO

- Ora i mix di tipo sono *giustamente* illegali: ☺

```
var res3 = Main.<String>find(arr1, new Frazione(4,6));
```

T = String | Array di stringhe | Elemento Frazione!? | Java

The parameterized method <String>find(String[], String) of type Main is not applicable for the arguments (String[], Frazione)

  - MOTIVO: invocando **`find`** con etichettatura **`<String>`**, il compilatore assume che sia **`T = String`**
  - di conseguenza, si aspetta un array di **`String`** e un **`elem`** di tipo **`String`**
  - trovando invece una **`Frazione`**, rileva l'errore ☺☺

# FUNZIONI GENERICHE: SINTASSI NEI VARI LINGUAGGI

DEFINIZIONE di funzione generica in tipo

Java

```
public static <T> boolean idem(T[] a, T[] b)
```

C#

```
public static bool idem<T>(T[] a, T[] b)
```

Scala

```
def idem[T](a: Array[T], b: Array[T]) : Boolean
```

Kotlin

```
public fun <T> idem(a: Array<T>, b: Array<T>):Boolean
```

- i vari linguaggi si differenziano per la posizione del segnaposto **`<T>`** (o **`[T]`** in Scala) e dei relativi dettagli sintattici

# FUNZIONI GENERICHE: SINTASSI NEI VARI LINGUAGGI

CHIAMATA di una funzione generica (con **T**=**Counter**)

Java

```
nomeclasse.<Counter>idem(array1, array2)
```

C#

```
nomeclasse.idem<Counter>(array1, array2)
```

Scala

```
nomeobject.idem(array1, array2)
```

Kotlin

```
nomeobject.idem(array1, array2)
```

OCCHIO: in Java, omettere l'indicazione del tipo attuale **<Counter>** fa sì che venga assunto **<Object>**, *disattivando così di fatto il controllo di tipo* !

- anche in fase di chiamata i vari linguaggi si differenziano per la posizione del segnaposto **<T>** (o **[T]**) e dei relativi dettagli sintattici

# BILANCIO

- Usare `Object` per ottenere genericità è un approccio *obsoleto* e *pericoloso*, perché *abolisce il controllo di tipo*
  - il «design intent» sottinteso di usare poi il «*giusto*» tipo in fase di chiamata non è catturato nel codice
  - è di fatto un vincolo «farlocco» ☹: a compile-time passa tutto, il contenuto può essere qualsiasi cosa
  - e quindi poi a run time, se qualcosa va storto… BOOM!
- Il tipo parametrico `<T>` esprime invece tutto in modo pulito
  - mix malsani di tipi diventano impossibili
  - la cura è *rafforzare* il controllo di tipo, *non eluderlo!*
  - il compilatore è un *alleato*, non un nemico!

# POLIMORFISMO VERTICALE vs POLIMORFISMO ORIZZONTALE

- L'approccio «old style» che sfrutta `Object` per ottenere genericità si chiama *polimorfismo verticale*
  - segue la gerarchia di ereditarietà
  - sembra semplice, e lo è…. in effetti, è *fin troppo semplice!*
- L'approccio basato sul tipo generico è invece spesso chiamato, per contraltare, *polimorfismo orizzontale*
  - opera "trasversalmente" rispetto alla gerarchia di ereditarietà
  - introduce il tipo come parametro
  - non influenza l'ereditarietà, non si basa su essa
  - è *l'approccio moderno* usato ormai a tappeto in tutti i linguaggi a oggetti con type system forti

# Un passo in più

# STRUTTURE DATI GENERICHE & COMPOSIZIONE

- Le strutture dati generiche supportano la *composizione*
  - ad esempio, potremmo definire uno **`Stack<T>`** implementandolo internamente con una **`List<T>`** usata come memoria operativa

Java

```
class Stack<T> {
    private List<T> memory;
    public Stack(){ memory = new List(); }
    public Stack<T> push(T e) {
            memory = new List(e, memory); return this; }
    public T pop() { var result = memory.head();
            memory = memory.tail(); return result; }
    public boolean isEmpty(){ return memory.isEmpty(); }
    public String toString(){ return memory.toString(); }
}
```

  - notare il trucchetto di restituire **`Stack<T>`** nella **`push`**, così che le relative chiamate possano essere concatenate → *fluent interface*

# STRUTTURE DATI GENERICHE & COMPOSIZIONE

- Esempio d'uso

Java

```
var stack = new Stack<String>();
stack.push("pippo").push("pluto").push("paperino");
System.out.println("stack = " + stack);
while(!stack.isEmpty()){
   System.out.println("pop: " + stack.pop());
   System.out.println("stack = " + stack);
}
```

```
stack = paperino, pluto, pippo
pop: paperino
stack = pluto, pippo
pop: pluto
stack = pippo
pop: pippo
stack =
```

# STRUTTURE DATI GENERICHE & EREDITARIETÀ

- Le strutture dati generiche supportano anche l'*ereditarietà*
  - ad esempio, potremmo definire **`ReversableList<T>`** come quella specializzazione di **`List<T>`** che offre anche **`reverse()`**
    - NB: ovviamente bisogna ridefinire pro forma i costruttori, in quanto, come sappiamo, non vengono ereditati

Java

```
class ReversableList<T> extends List<T> {
    public ReversableList(){ super();}
    public ReversableList(T e){ super(e); }
    public ReversableList(T e, List<T> l){ super(e,l); }
    public ReversableList reverse() {…}
}
```

  - DUBBIO: come definire il metodo **`reverse()`**?

# STRUTTURE DATI GENERICHE & EREDITARIETÀ

- Per definire **`reverse`** occorre distinguere:
  - se la lista è vuota o comprende un solo elemento, il risultato di **`reverse()`** è, ovviamente, la lista stessa
  - altrimenti, occorre scorrere la lista data ricorsivamente, costruendo ad ogni passo una seconda lista ausiliaria *che abbia in testa l'elemento tolto dall'altra*: in tal modo, gli elementi nella lista ausiliaria risulteranno in ordine inverso rispetto all'originale
  - ESEMPIO: lista data = [«Aaa», «Bbb», «Ccc»]

    1° passo: testa = «Aaa», coda = [ «Bbb», «Ccc»], aux = [«Aaa»]

    2° passo: testa = «Bbb», coda = [ «Ccc»], aux = [«Bbb», «Aaa»]

    3° passo: testa = «Ccc», coda = [ ], aux = [«Ccc», «Bbb», «Aaa»]
  - per farlo, conviene predisporre due metodi: il vero **`reverse()`** senza argomenti, e un **`reverse()`** ausiliario a due argomenti che gestisca il ciclo ricorsivo

# STRUTTURE DATI GENERICHE & EREDITARIETÀ

Java

```
class ReversableList<T> extends List<T> {
   public ReversableList(){ super();}
   public ReversableList(T e){ super(e); }
   public ReversableList(T e, List<T> l){ super(e,l); }
   public ReversableList reverse() {
      if(this.isEmpty() ||
          (!this.isEmpty() && this.tail().isEmpty()) )
        return this;
      else
        return reverse(this, new ReversableList<T>());
   }
   private ReversableList<T> reverse(List<T> me,
                                     ReversableList<T> acc) {
      if(me==null) return acc;
      else return reverse(me.tail(),
                  new ReversableList(me.head(), acc));
    }
}
```

# STRUTTURE DATI GENERICHE & EREDITARIETÀ

- Esempio d'uso

Java

```
var rl =
   new ReversableList<String>("Piero",
     new ReversableList<String>("Alice",
       new ReversableList<String>("Giovanni") ));
System.out.println("lista = " + rl);
System.out.println("reversed = " + rl.reverse());
```

```
lista = Piero, Alice, Giovanni
reversed = Giovanni, Alice, Piero
```

# CONCLUSIONE

- Scrivere funzioni o strutture dati *generiche in tipo* è possibile
- Il modo «old style» di farlo è etichettare gli argomenti come `Object` → *polimorfismo verticale*
  - di fatto abolisce il controllo di tipo
  - i *mis-use case sfuggono a compile time ed esplodono a run time*
- Il modo moderno è adottare il *tipo parametrico (generico)* → *polimorfismo orizzontale*
  - rende il tipo stesso un (ulteriore) parametro
  - rafforza il controllo di tipo
  - intercetta i mis-use case a *compile time*
- Ne riparleremo in futuro, parlando di *varianza*